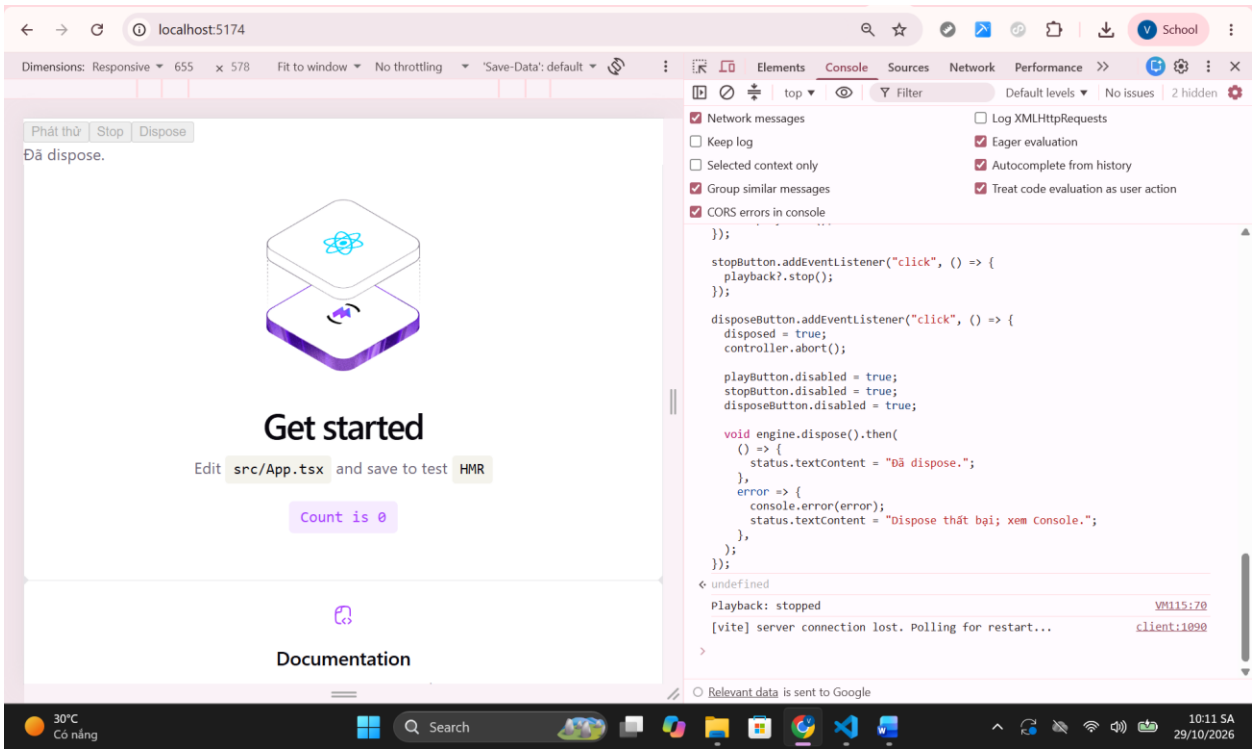
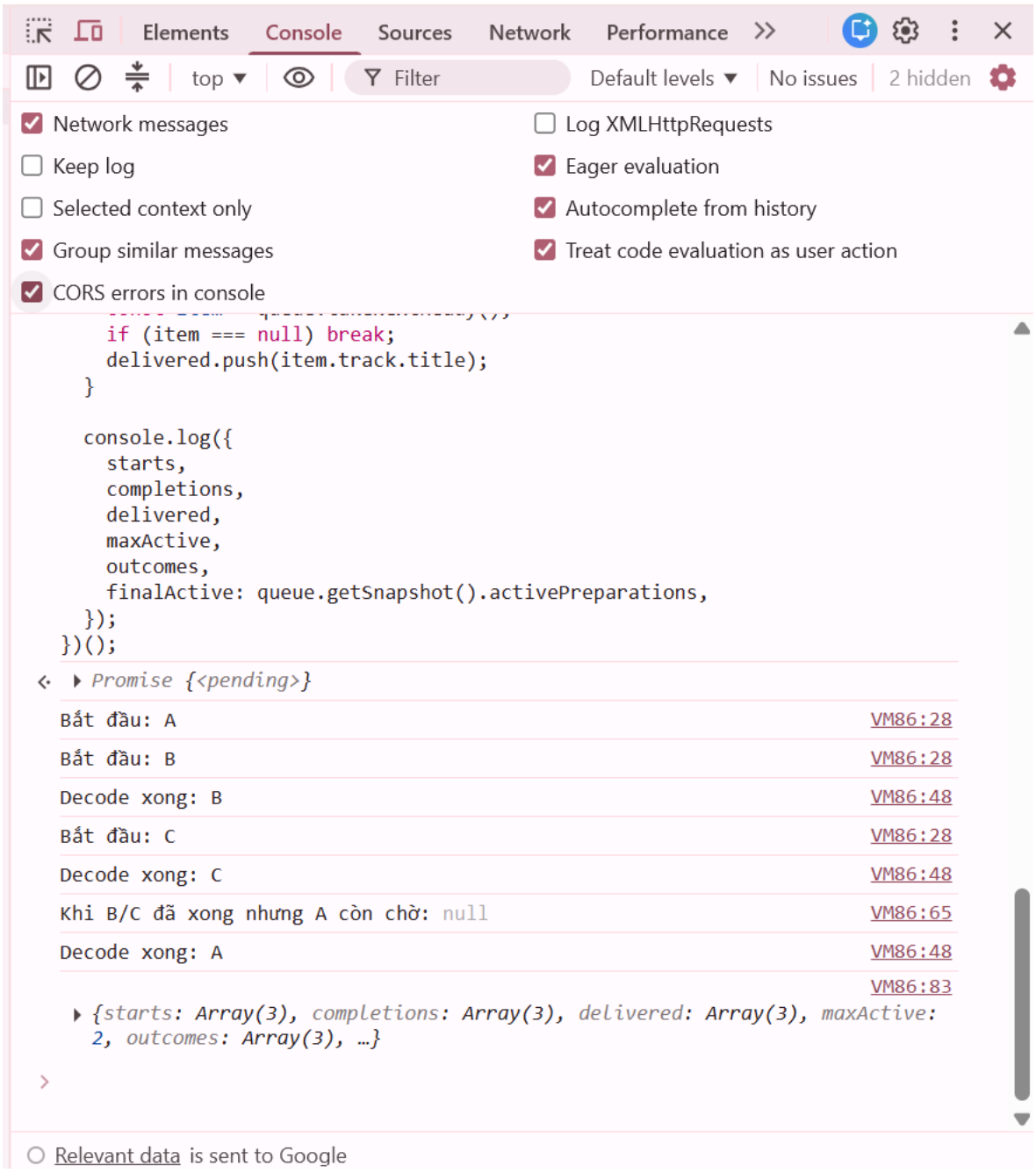
HW2:

1. Test audio(play,stop.dispose) in Audio Enginee

Result: no error in console, when click “phát thử”, mp3 can be started.



2.FIFO



Testing failure decode: When A error, but continues to B and C.

```
  );

  const delivered = [];

  for (;;) {
    const item = queue.takeNextReady();
    if (item === null) break;
    delivered.push(item.track.title);
  }

  console.log(
    "Trạng thái:",
    outcomes.map(outcome => outcome.status),
  );
  console.log("Thứ tự lấy:", delivered);
  console.log(
    "Số tác vụ đang chạy:",
    queue.getSnapshot().activePreparations,
  );
  console.log("Lấy thêm:", queue.takeNextReady());
})().catch(console.error);
```

```
▸ Promise {<pending>}
Trạng thái: ▸ (3) ['error', 'ready', 'ready']   VM96:44
Thứ tự lấy: ▸ (2) ['B', 'C']                    VM96:48
Số tác vụ đang chạy: 0                          VM96:49
Lấy thêm: null                                  VM96:53
>
```

Relevant data is sent to Google

2. Testing manual

```
async () => {
 const { createPreparationQueue } = await import(
  "/src/queue/preparation-queue.ts"
 );


 function deferred() {
  let resolve;
  let reject;


  const promise = new Promise((res, rej) => {
```

```
    resolve = res;
    reject = rej;
  });

  return { promise, resolve, reject };
}

function makeHarness() {
  const fetches = new Map();
  const decodes = new Map();

  const queue = createPreparationQueue({
    fetchAudio: (title, signal) => {
      const gate = deferred();
      fetches.set(title, { gate, signal });

      const abort = () => {
        gate.reject(
          new DOMException("Fetch bị hủy.", "AbortError"),
        );
      };

      if (signal.aborted) {
        abort();
      } else {
        signal.addEventListener("abort", abort, { once: true });
```

```
    }

    return gate.promise.finally(() => {
      signal.removeEventListener("abort", abort);
    });
  },

  engine: {
    decode: bytes => {
      const title = String.fromCharCode(
        new Uint8Array(bytes)[0],
      );

      const gate = deferred();
      decodes.set(title, gate);

      // Cố ý không hỗ trợ AbortSignal.
      return gate.promise;
    },
  },
});

async function waitUntil(check) {
  for (let count = 0; count < 100; count += 1) {
    if (check()) return;
    await new Promise(resolve => setTimeout(resolve, 0));
```

```
  }

  throw new Error("Không đạt điều kiện chờ kiểm tra.");
}

async function finishFetch(title) {
  await waitUntil(() => fetches.has(title));

  fetches.get(title).gate.resolve(
    new Response(
      new Uint8Array([title.charCodeAt(0)]),
      { status: 200 },
    ),
  );

  await waitUntil(() => decodes.has(title));
}

function finishDecode(title) {
  decodes.get(title).resolve(
    new AudioBuffer({
      numberOfChannels: 1,
      length: 1,
      sampleRate: 44100,
    }),
  );
```

```
  }

  function add(title) {
    return queue.enqueue({ title, url: title });
  }

  return {
    queue,
    fetches,
    decodes,
    waitUntil,
    finishFetch,
    finishDecode,
    add,
  };
}

// 1. Hủy khi queued: C chưa được tải.
{
  const h = makeHarness();
  const a = h.add("A");
  const b = h.add("B");
  const c = h.add("C");

  console.log("queued: lần hủy đầu", c.cancel());
  console.log("queued: hủy lặp", c.cancel());
```

```
  console.log("queued: outcome", await c.prepared);

  await h.waitUntil(() => h.fetches.size === 2);

  console.log("queued: C chưa fetch", !h.fetches.has("C"));

  await h.queue.dispose();
  await Promise.all([a.prepared, b.prepared]);

  console.log(
    "queued: active cuối",
    h.queue.getSnapshot().activePreparations,
  );
}

// 2. Hủy trong fetch; queue tiếp tục nhận C.
{
  const h = makeHarness();
  const a = h.add("A");
  const b = h.add("B");
  const c = h.add("C");

  await h.waitUntil(() => h.fetches.has("A"));

  a.cancel();
```

```
  console.log(
    "fetch: signal aborted",
    h.fetches.get("A").signal.aborted,
  );

  console.log("fetch: outcome", await a.prepared);

  await h.waitUntil(() => h.fetches.has("C"));

  console.log("fetch: C đã bắt đầu", true);

  await h.queue.dispose();
  await Promise.all([b.prepared, c.prepared]);
}

// 3. Hủy trong decode, vẫn giữ slot.
{
  const h = makeHarness();
  const a = h.add("A");
  const b = h.add("B");
  const c = h.add("C");

  await h.finishFetch("A");

  a.cancel();
```

```
console.log("decode: outcome", await a.prepared);

console.log(
  "decode: active vẫn là 2",
  h.queue.getSnapshot().activePreparations,
);

console.log(
  "decode: C chưa bắt đầu",
  !h.fetches.has("C"),
);

// Decode A thực sự kết thúc tại đây.
h.finishDecode("A");

await h.waitUntil(() => h.fetches.has("C"));

console.log(
  "decode: A không trở lại ready",
  h.queue.getSnapshot().tasks.find(t => t.id === a.id).status,
);

console.log("decode: C chỉ bắt đầu sau khi A xong", true);

await h.queue.dispose();
```

```
  await Promise.all([b.prepared, c.prepared]);
}

// 4. Dispose khi A decode, B fetch và C queued.
{
  const h = makeHarness();
  const a = h.add("A");
  const b = h.add("B");
  const c = h.add("C");

  await h.finishFetch("A");

  let disposeFinished = false;

  const first = h.queue.dispose();
  const second = h.queue.dispose();

  first.then(() => {
    disposeFinished = true;
  });

  console.log("dispose: cùng promise", first === second);

  const outcomes = await Promise.all([
    a.prepared,
    b.prepared,
```

```
  c.prepared,
]);

console.log(
  "dispose: task outcomes",
  outcomes.map(x => x.status),
);

console.log(
  "dispose: B fetch bị abort",
  h.fetches.get("B").signal.aborted,
);

console.log(
  "dispose: C chưa từng fetch",
  !h.fetches.has("C"),
);

console.log(
  "dispose: vẫn chờ decode A",
  !disposeFinished,
);

// Kết quả muộn không được ghi vào queue.
h.finishDecode("A");
await first;
```

```
  console.log("dispose: snapshot cuối", h.queue.getSnapshot());
  console.log("dispose: không lấy được buffer", h.queue.takeNextReady());

  try {
    h.add("D");
    console.error("Không đạt: enqueue sau dispose được chấp nhận.");
  } catch (error) {
    console.log("dispose: enqueue bị từ chối", error.name);
  }
}

// 5. Hủy ready trước khi bàn giao.
{
  const h = makeHarness();
  const a = h.add("A");

  await h.finishFetch("A");
  h.finishDecode("A");

  console.log("ready: outcome ban đầu", await a.prepared);
  console.log("ready: hủy", a.cancel());
  console.log("ready: hủy lặp", a.cancel());
  console.log("ready: không bàn giao", h.queue.takeNextReady());

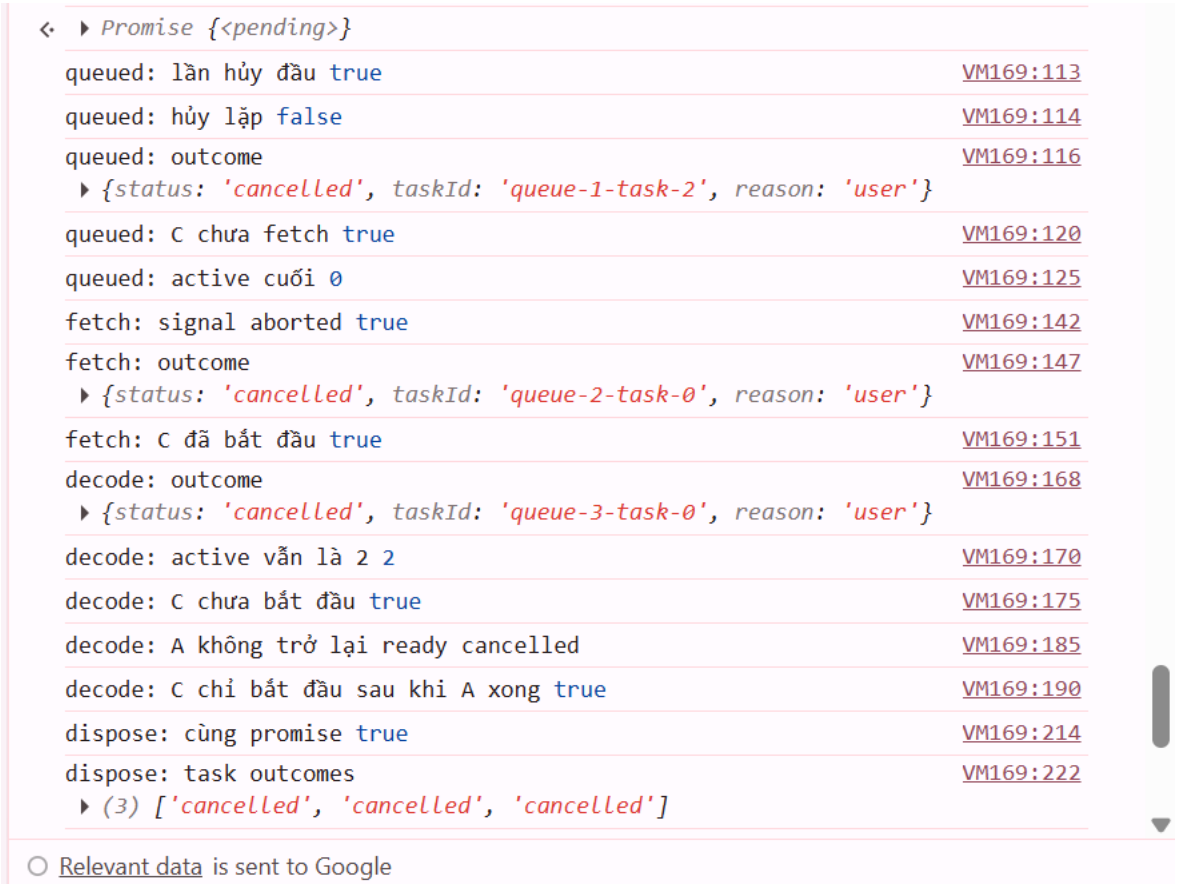
  await h.queue.dispose();
```

```
}

 console.log("Đã chạy hết kịch bản; đối chiếu kết quả trước khi đánh dấu đạt.");

})().catch(console.error);
```

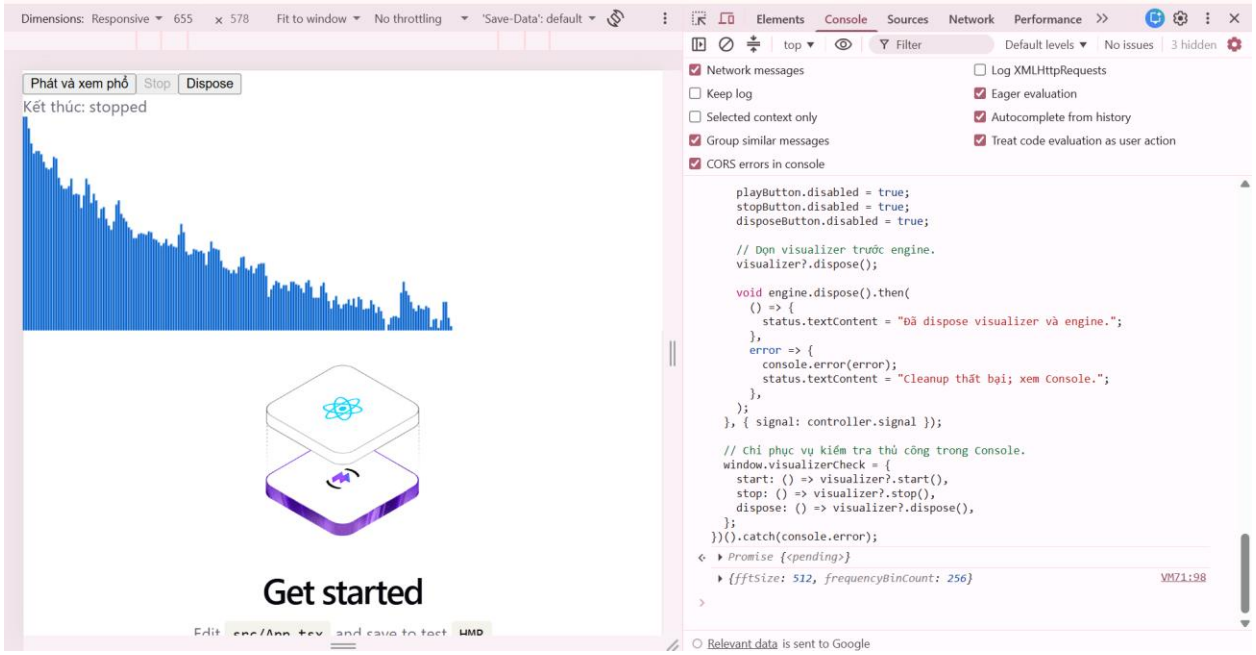
Result:



```
dispose: task outcomes                                          VM169:222
 ▸ (3) ['cancelled', 'cancelled', 'cancelled']
dispose: B fetch bị abort true                                  VM169:227
dispose: C chưa từng fetch true                                 VM169:232
dispose: vẫn chờ decode A true                                  VM169:237
dispose: snapshot cuối                                          VM169:246
 ▸ {disposed: true, activePreparations: 0, tasks: Array(0)}
dispose: không lấy được buffer null                             VM169:247
dispose: enqueue bị từ chối InvalidStateError                   VM169:253
ready: outcome ban đầu                                          VM169:265
 ▸ {status: 'ready', taskId: 'queue-5-task-0'}
ready: hủy true                                                 VM169:266
ready: hủy lặp false                                            VM169:267
ready: không bàn giao null                                      VM169:268
Đã chạy hết kịch bản; đối chiếu kết quả trước khi đánh dấu đạt.  VM169:273
>
```

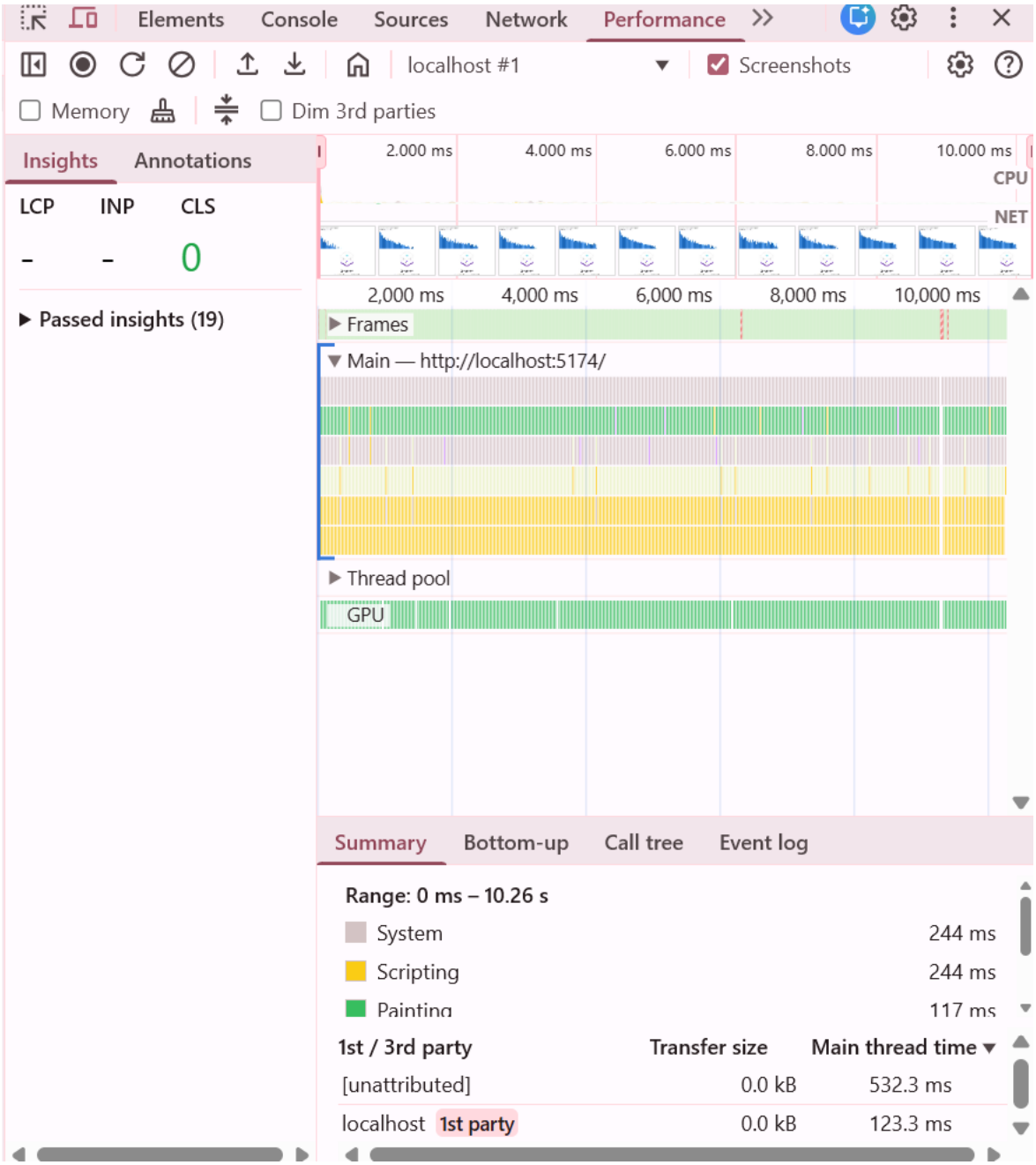
Relevant data is sent to Google

4.Visualization audio

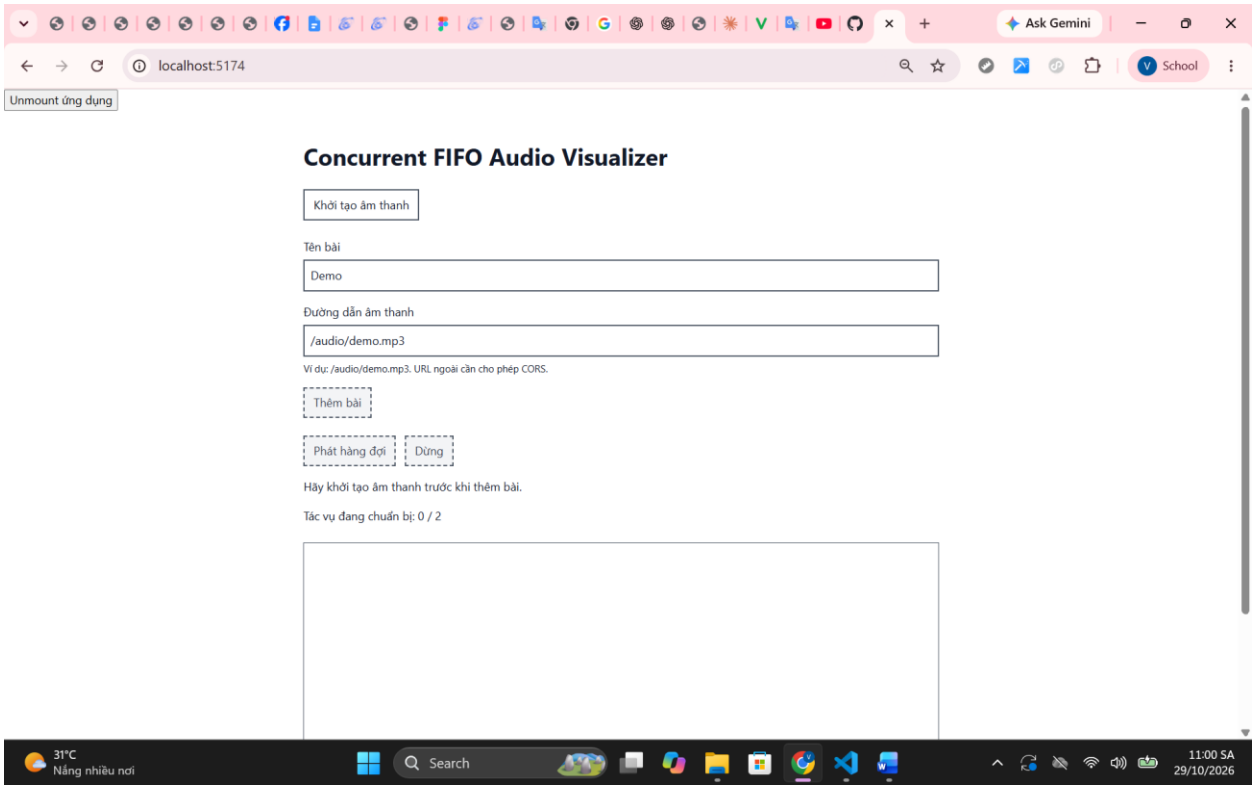


Record and check visual performance

Result: not 60 FPS



3. User Interface for web


Ask Gemini
localhost:5174
School
Unmount ứng dụng
Concurrent FIFO Audio Visualizer
Khởi tạo âm thanh
Tên bài
Demo
Đường dẫn âm thanh
/audio/demo.mp3
Ví dụ: /audio/demo.mp3. URL ngoài cần cho phép CORS.
Thêm bài
Phát hàng đợi
Dừng
Hãy khởi tạo âm thanh trước khi thêm bài.
Tác vụ đang chuẩn bị: 0 / 2
31°C
Nắng nhiều nơi
Search
11:00 SA
29/10/2026